

Glassdoor Reviews NLP

Varun Nadgir

METCS 688

Introduction

- Data:
 - <https://www.kaggle.com/datasets/davidgauthier/glassdoor-job-reviews>
- Columns of interest include:
 - Name of Company
 - Job Title of Reviewer
 - Pros of Job
 - Cons of Job
 - and some others
- Used Google Colab and then mounted Google Drive pointing to where the .csv was kept

Introduction

- In this project, we will:
 - Extract keywords using KeyBERT based on various companies & job titles
 - Try to correctly label job title or company using spaCy text categorizer built on BERT sentence transformer
- We will see how the parameters of these models work
- Explore some possible future ideas with this dataset (or one similar)

Introduction

Some preprocessing:

```
# strip whitespace at beginning and end of job title
```

```
df['job_title'] = df['job_title'].apply(lambda x: x.strip())
```

```
# replace blank job titles with na
```

```
df['job_title'] = df['job_title'].replace(r'^\s*$', np.nan, regex=True)
```

```
# drop rows with missing job title
```

```
df = df.dropna(subset=['job_title'])
```

```
# add new column containing full review text as one body
```

```
df['fullreview'] = df['pros'] + ' ' + df['cons']
```

Introduction

- Quick look at `df.head()`

	firm	date_review	job_title	current	location	overall_rating	work_life_balance	culture_values	diversity_inclusion	career_opp	comp_benefits	senior_mgmt	recommend	ceo_approv	outlook	headline	pros	cons	fullreview
1	AFH-Wealth-Management	2015-12-11	Office Administrator	Current Employee, more than 1 year	Bromsgrove, England, England	2	3.0	1.0	NaN	2.0	1.0	4.0	x	o	r	Excellent staff, poor salary	Friendly, helpful and hard-working colleagues	Poor salary which doesn't improve much with pr...	Friendly, helpful and hard-working colleagues ...
2	AFH-Wealth-Management	2016-01-28	Office Administrator	Current Employee, less than 1 year	Bromsgrove, England, England	1	1.0	1.0	NaN	1.0	1.0	1.0	x	o	x	Low salary, bad micromanagement	Easy to get the job even without experience in...	Very low salary, poor working conditions, very...	Easy to get the job even without experience in...
4	AFH-Wealth-Management	2016-04-23	Office Administrator	Current Employee, more than 1 year	Bromsgrove, England, England	1	2.0	1.0	NaN	2.0	1.0	1.0	x	o	x	client reporting admin	Easy to get the job, Nice colleagues.	Abysmal pay, around minimum wage. No actual tr...	Easy to get the job, Nice colleagues. Abysmal ...
5	AFH-Wealth-Management	2016-05-26	Office Administrator	Current Employee, less than 1 year	NaN	3	4.0	2.0	NaN	2.0	3.0	2.0	o	r	r	Office administrator	Some good people to work with.\n\nFlexible wor...	Morale.\n\nLack of managerial structure.\n\nDo...	Some good people to work with.\n\nFlexible wor...
6	AFH-Wealth-Management	2016-09-23	IFA	Former Employee	Bromsgrove, England, England	1	1.0	1.0	NaN	1.0	1.0	1.0	x	o	r	It horrible management	Good investment management strategy. Overall t...	The management and seniors are ruthless. No tr...	Good investment management strategy. Overall t...

Keyword Extraction

see which companies have highest review counts

```
print(df['firm'].value_counts().head(15))
```

- Most frequently reviewed companies
- We will focus on just a few for now – McDonald's, Google, and Apple

IBM	54153
McDonald-s	45009
Deloitte	42624
EY	30753
PwC	29581
Oracle	28878
Microsoft	23371
J-P-Morgan	22786
KPMG	22308
Apple	18573
Citi	16215
Google	14111
SAP	13102
HSBC-Holdings	12258
Tesco	11552

Name: firm, dtype: int64

Keyword Extraction

instantiate basic BERT model

```
kw_model = KeyBERT()
```

instantiate BERT model s using Sentence Transformer base

```
sentence_model = SentenceTransformer('paraphrase-MiniLM-L6-v2')
```

```
kw_model_s = KeyBERT(model=sentence_model)
```

Keyword Extraction

- Subset for McDonald's and check `value_counts()` for job titles
- Seems to be some overlap or redundancy in naming
- Let's dig into Cashier and Manager as their job responsibilities should be somewhat different

```
Anonymous Employee      8537
Crew Member             7690
Cashier                 4910
Mcdonalds Crew Member  2028
Crew Trainer            1317
Manager                 1111
Shift Manager           959
Cook                    877
Team Member             718
Line Cook                587
Name: job_title, dtype: int64
```

Keyword Extraction

- Two methods:
- Maxsum
 - First take ($2 \times \text{top_n}$) most similar words/phrases to the entire document
 - Then take top_n combinations, extracting those that are least similar to each other by cosine similarity
- Maximal Marginal Relevance (MMR):
 - Considers similarity of keywords/phrases to the document while also considering similarity to already selected keywords/phrases
 - Can manage this strictness by using its diversity parameter

<https://maartengr.github.io/KeyBERT/api/maxsum.html>

<https://maartengr.github.io/KeyBERT/api/mmr.html>

Keyword Extraction

```
# create documents of pros and cons, filtered by job title
```

```
mcdonalds_df = df[df['firm'] == 'McDonald-s']
```

```
# cashier
```

```
mcd_cashier_pros = ''.join(mcdonalds_df[mcdonalds_df['job_title'] == 'Cashier']['pros'].sample(n=1000, replace=False))
```

```
mcd_cashier_cons = ''.join(mcdonalds_df[mcdonalds_df['job_title'] == 'Cashier']['cons'].sample(n=1000, replace=False))
```

```
# manager
```

```
mcd_manager_pros = ''.join(mcdonalds_df[mcdonalds_df['job_title'] == 'Manager']['pros'].sample(n=1000, replace=False))
```

```
mcd_manager_cons = ''.join(mcdonalds_df[mcdonalds_df['job_title'] == 'Manager']['cons'].sample(n=1000, replace=False))
```

Keyword Extraction

```
kw_model.extract_keywords(  
    mcd_cashier_pros,  
    keyphrase_ngram_range=(3,5),  
    stop_words='english',  
    use_maxsum=True,  
    nr_candidates=20,  
    top_n=5  
)
```

```
[('work great coworkers friendly', 0.5138),  
 ('awesome place work college', 0.5205),  
 ('student job decent hours good', 0.5206),  
 ('started mcdonald great experience workplace', 0.5218),  
 ('food great awesome nice salary', 0.5282)]
```

```
kw_model.extract_keywords(  
    mcd_cashier_pros,  
    keyphrase_ngram_range=(3,5),  
    stop_words='english',  
    use_mmr=True,  
    diversity=0.7  
)
```

```
[('awesome place work college student', 0.5645),  
 ('manager structured easy', 0.2186),  
 ('pace average pay', 0.1376),  
 ('food quick leave early pros', 0.1291),  
 ('ok ok ok friendly people', 0.1282)]
```

Keyword Extraction

Cashier Cons (Maxsum):

```
[('customers distasteful low hours work', 0.565),  
 ('stay hours long customers hard', 0.5695),  
 ('customer service job abysmal management', 0.5769),  
 ('shifts poor management rude customers', 0.5801),  
 ('restaurant occasionally deal difficult customers', 0.6021)]
```

Cashier Cons (MMR):

```
[('restaurant occasionally deal difficult customers', 0.6021),  
 ('progress career long hours', 0.2668),  
 ('environment mcflurry machine', 0.1203),  
 ('oh wasn bad terrible', 0.106),  
 ('general public salary', 0.0954)]
```

Keyword Extraction

Manager Pros (Maxsum):

```
[('decent coworkers free meals flexibility', 0.5637),  
 ('free food good young teammates', 0.5665),  
 ('amazing food fun work environment', 0.5677),  
 ('treat guests good job teen', 0.5744),  
 ('working friends customers great food', 0.576)]
```

Manager Pros (MMR):

```
[('good food good working experience', 0.5913),  
 ('dealing customers free', 0.2736),  
 ('people young age', 0.2707),  
 ('scheduling school tuition help', 0.1896),  
 ('setting easy going flexible', 0.1672)]
```

Keyword Extraction

Manager Cons (Maxsum):

```
[('typically staff bad working environment', 0.5337),  
 ('job bad customers long hours', 0.5399),  
 ('long hours management sucked organizations', 0.5446),  
 ('lacking good having workers cons', 0.5558),  
 ('bad pay management great cons', 0.5577)]
```

Manager Cons (MMR):

```
[('comparatively pay busy workplace bad', 0.6006),  
 ('care company managers abusive younger', 0.3384),  
 ('restaurant ensuring safety', 0.2089),  
 ('cons tbh long', 0.1886),  
 ('rest crew replacement embrace', 0.0659)]
```

Keyword Extraction

- Followed the same procedure as last example, except this time comparing Pros & Cons of the same job title but from different companies

- Software Engineer

- Google

Anonymous Employee	3126
Software Engineer	2096
Senior Software Engineer	452
Account Manager	222
Product Manager	215
Software Developer	210
Program Manager	174
Account Strategist	155
Intern	138
Analyst	135
Name: job_title, dtype: int64	

Apple	Anonymous Employee	3604
	Specialist	1875
	Technical Specialist	555
	Software Engineer	400
	Genius	377
	Mac Specialist (Apple Store)	365
	Apple Genius	333
	Mac Genius	331
	Family Room Specialist	311
	Apple At Home Advisor	305
Name: job_title, dtype: int64		

Keyword Extraction

Google Pros (Maxsum):

```
[('work big company like', 0.6308),  
 ('best industry work life balance', 0.6308),  
 ('lot benefits google management engineer', 0.6389),  
 ('products large scale work life', 0.6604),  
 ('good culture working google company', 0.6786)]
```

Google Pros (MMR):

```
[('work big company like google', 0.6953),  
 ('happiness result happy', 0.3192),  
 ('environment life time', 0.2735),  
 ('food stock maternity stuff', 0.1737),  
 ('free lessons best', 0.0333)]
```

Apple Pros (Maxsum):

```
[('benefits innovative company lot apple', 0.6912),  
 ('apple really good company', 0.6914),  
 ('apple great company incredibly friendly', 0.6928),  
 ('busy work apple good reasons', 0.7065),  
 ('enjoy job apple innovative changes', 0.7132)]
```

Apple Pros (MMR):

```
[('really enjoy job apple innovative', 0.7787),  
 ('compensation package level balanced', 0.131),  
 ('ecosystem functions fairly inner turmoil', 0.0511),  
 ('latest wireless technologies lte', 0.0211),  
 ('people death marches', -0.1342)]
```

Keyword Extraction

Google Cons (Maxsum):

```
[('start getting promotion hard', 0.6046),  
 ('difficult promotion frustrated work late', 0.6125),  
 ('promotion process far', 0.6135),  
 ('think cons slow promotion process', 0.6275),  
 ('careful getting promoted growing career', 0.6277)]
```

Google Cons (MMR):

```
[('difficult promotion process', 0.6474),  
 ('feels like huge', 0.1698),  
 ('ll retire 401', 0.0867),  
 ('bug fixes core', 0.0295),  
 ('depending location remote', -0.083)]
```

Apple Cons (Maxsum):

```
[('doesn better work life', 0.5577),  
 ('stressful level use contractors really', 0.5607),  
 ('environment hard work people', 0.5617),  
 ('workplaces crazy stuff', 0.5689),  
 ('feel unappreciated hours likely work', 0.572)]
```

Apple Cons (MMR):

```
[('makes work life balance complicated', 0.5842),  
 ('launches good couches cold', 0.212),  
 ('sponsorship secrecy tight won', 0.164),  
 ('offshore india completely different timezone', 0.154),  
 ('weed leakers expect', 0.0655)]
```

Text Classification

- Subset for 3 different job titles – Manager, Software Engineer, Cashier
- See if a text classifier can correctly identify what job the review text is about
- Initial prediction is that Cashier and Manager may be easily confused, but Software Engineer should be clear

Text Classification

```
# create non-overlapping sets for training and testing
```

```
manager_train = manager_df.sample(n=500, replace=False)
```

```
manager_test = manager_df[~manager_df['fullreview'].isin(manager_train['fullreview'])].sample(n=100, replace=False)
```

```
# same for engineer & cashier
```

```
# create dictionary which will train model
```

```
data = {  
    "manager": list(manager_train['fullreview']),  
    "engineer": list(engineer_train['fullreview']),  
    "cashier": list(cashier_train['fullreview'])  
}
```

Text Classification

```
# create text categorizer using huggingface sentence transformer model
nlp = spacy.blank("en")
nlp.add_pipe(
    "text_categorizer",
    config={
        "data": data,
        "model": "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2",
        "device": "cpu"
    }
)
```

Text Classification

```
# apply model to the review text of the test datasets
```

```
manager_test['manager_score'], manager_test['engineer_score'], manager_test['cashier_score'] =  
    zip(*manager_test['fullreview'].apply(lambda x: (nlp(x)._.cats['manager'], nlp(x)._.cats['engineer'], nlp(x)._.cats['cashier'])))
```

- We can create new columns for the category weights provided by `nlp()._.cats`
- Wrote custom function that checks which score is the highest and declares that as the “prediction”

	job_title	fullreview	manager_score	engineer_score	cashier_score	predicted
1738	Manager	Some great people in branches and some good di...	0.132225	0.345707	0.522068	Cashier
7770	Manager	Experience, technical skills, poactive, human ...	0.377533	0.103239	0.519228	Cashier
523	Manager	Focussed on people\r\nEmpowering\r\nInterestin...	0.072562	0.060443	0.866995	Cashier
15137	Manager	Great people across the globe, a huge variety ...	0.023835	0.215418	0.760746	Cashier
10223	Manager	Good social employee and you can't get bored a...	0.162083	0.570702	0.267215	Software Engineer

Text Classification

- Overall results seemed to align with prediction

- Manager Test:

```
Cashier          61
Software Engineer 27
Manager          12
Name: predicted, dtype: int64
```

- Software Engineer Test:

```
Software Engineer 64
Cashier           30
Manager           6
Name: predicted, dtype: int64
```

- Cashier Test:

```
Manager          80
Cashier           11
Software Engineer 9
Name: predicted, dtype: int64
```

Text Classification

- Followed the same procedure as last example, except this time to see if Software Engineer review text can be used to identify which company
- Not sure how to predict this one might perform, as many of the words would be reused and the overall experience would be similar

create dictionary which will train model

```
data_firm = {  
    "google": list(google_train['fullreview']),  
    "apple": list(apple_train['fullreview']),  
    "msoft": list(msoft_train['fullreview'])  
}
```

Text Classification

- Oddly similar results to the previous example:

- Google Test:

```
Apple      53
Google     27
Microsoft  20
Name: predicted, dtype: int64
```

- Apple Test:

```
Google     50
Microsoft  27
Apple      23
Name: predicted, dtype: int64
```

- Microsoft Test:

```
Microsoft  58
Google     23
Apple      19
Name: predicted, dtype: int64
```

Conclusion

- KeyBERT let us find keywords for various jobs within a company, as well as the same job title in different companies
 - Personally preferred maxsum method for this dataset, as the results were more coherent even though some concepts were repeated
- spaCy let us attempt to label a review as a particular job title or company based on its contents
 - Could have improved this by being more particular in how the dictionary which trained the model was created
 - Using full bodies of text seemed work against the quality of results due to lots of repeated words and concepts making it hard to recognize distinct features
 - With more processing power, would be a good experiment to create the training dictionary by extracting long, unique keyphrases with KeyBERT
- Overall, results were very interesting and performed better than expected



Questions?



Thank You!